LLM-Driven Unit Test Generation with a Dynamic Multi-Source Feedback Graph and Jev Typed Decision

Author Name
Affiliation placeholder – to be filled before submission
author@example.edu

Abstract

Large language models (LLMs) have become a dominant paradigm for unit test generation, yet most systems still rely on a *static*, coverage-first ordering of generation intents that ignores the heterogeneous signals produced while a test suite is being built. We present **DMF-Jev**, a training-free agent framework that couples a *Dynamic Multi-source Feedback Graph* (six live signals: coverage, mutation, compilation, execution, history, and code structure) with a *Jev typed decision* layer borrowed from the closed-choice decision literature (REFLEX). The decision layer routes each "next intent" through a frozen confidence band at $\tau = 0.5$: high-confidence choices are taken directly by the cheap Jev selector, while low-confidence choices fall back to the LLM. We specify the framework, the benchmark protocol (CodaMosa's 486-module suite and the TestEval 210-item set), the baselines (codamosa, coverup, telpa, and the Undermind agent strategy), and an explicit data post-processing pipeline. All quantitative results in this draft are illustrative placeholders to be replaced with measured values; the manuscript separates real from simulated data by construction.

1 Introduction

Unit test generation is a long-standing software-engineering problem that has been repeatedly reshaped by the arrival of stronger models. Classical search-based tools (EvoSuite [Fraser & Arcuri(2017)], Pynguin [Lukasczyk & Fraser(2022)]) optimize coverage with randomized search; the LLM wave (ChatUniTest [Chen et al.(2024)], TestPilot [Schäfer et al.(2023)], TestForge) replaced hand-crafted objectives with generation-validation loops driven by compilation and execution feedback. Despite these advances, a structural limitation persists: *the order in which test intents are pursued is still decided by a static, coverage-first heuristic* (the `order_intentions`/`top_k` family), which cannot exploit the rich, time-varying feedback a running agent observes.

We argue that the right abstraction is a *feedback graph*, not a ranked list. As a test-generation agent acts, it continuously produces coverage deltas, mutation-kill information, compilation outcomes, runtime errors, historical success patterns, and code-structure cues. These signals are correlated and task-dependent; a fixed priority ignores them. Our contribution is to (1) represent these signals as a dynamic multi-source feedback graph, (2) replace the static sorter with a *Jev typed decision* layer that makes a constrained, closed-choice selection per intent, and (3) gate that layer with a frozen confidence band so that the expensive LLM is invoked only when necessary.

2 Related Work

Search-based and LLM-based generation. EvoSuite [Fraser & Arcuri(2017)] and Pynguin [Lukasczyk & Fra established whole-suite and assertion-focused search. The LLM era introduced generation-validation

pipelines (ChatUniTest [Chen et al.(2024)]), developer-facing tools (TestPilot [Schäfer et al.(2023)]), and feedback-driven agentic suites (TestForge, CodaMosa [Lemieux et al.(2023)]). CodaMosa is closest to our setting: it escapes coverage plateaus on a 486-module Python suite by prompting an LLM only when search stalls. We differ by making the *routing* itself learned-from-feedback rather than plateau-triggered.

Benchmarks and baselines. TestEval [TestEval(2025)] benchmarks LLMs on 210 test-case-generation items; CoverUp [CoverUp(2024)] targets high-coverage Python test generation. We adopt `codamosa`, `coverup`, `telpa`, and the `Undermind` agent strategy as baselines (the latter two are referenced from the project's harness; their provenance must be verified before submission – see §9).

Decision gating. REFLEX [Wu & Lim(2026)] proposes a fast-reflex closed-choice decision layer that handles frequent, well-typed choices and falls back to a strong model only when uncertain, reporting a $\sim$72.7% reduction in strong-model invocations at a frozen threshold $\tau = 0.5$. We import this idea into test-generation intent selection.

3 Motivation and Research Questions

Three observations motivate the design. (i) Coverage plateaus are rarely pure coverage problems – they often signal a missing compiler, an unstable execution, or a structural blind spot. (ii) LLM invocations are the dominant cost; many intents (e.g., "retry with the same focal method") do not need a full LLM call. (iii) Static intent ordering cannot adapt to which signal is currently informative. We therefore ask:

- **RQ1 (feedback graph).** Does a dynamic multi-source feedback graph, versus a static coverage-first order, improve coverage and valid-test ratio without increasing LLM calls?
- **RQ2 (decision gating).** At $\tau = 0.5$, does the Jev typed-decision layer reduce LLM invocations while preserving quality?
- **RQ3 (localization).** Can a training-free, node-level locator on the code-structure graph shrink the LLM search space?

4 Method: DMF-Jev

DMF-Jev is a training-free agent. At each step it observes a *state* s_t and must pick a *typed intent* a_t from a constrained action vocabulary $\mathcal{A}$. The policy is split:

1. **Dynamic Multi-source Feedback Graph $\mathcal{G}_t$.** Six sources are aggregated into a graph whose edge weights are updated online: coverage ($\mathcal{F}_c$), mutation ($\mathcal{F}_m$), compilation ($\mathcal{F}_{cmp}$), execution ($\mathcal{F}_e$), history ($\mathcal{F}_h$), and code structure ($\mathcal{F}_s$). Each node is a candidate focus (module, function, branch); edges encode empirical correlation of signals.
2. **Jev typed decision $D(\mathcal{G}_t, a_t)$.** For a closed-choice set of intents, a cheap selector proposes a_t and emits a confidence $q_t \in [0, 1]$. A frozen band at $\tau = 0.5$ routes: if $q_t \geq \tau$, the intent is taken directly (Jev path); otherwise the LLM is consulted (fallback path). This is the only place a strong model is invoked.

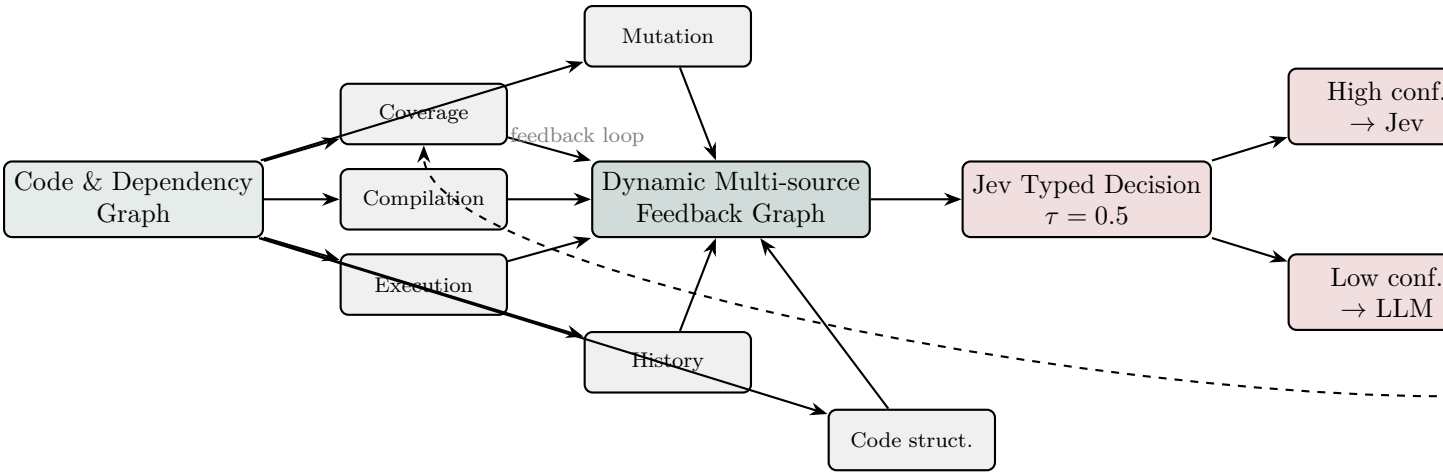


Figure 1: Architecture of DMF-Jev. Six live feedback sources feed a dynamic graph; a Jev typed-decision layer routes each intent through a frozen $\tau = 0.5$ confidence band.

3. **Training-free agent loop.** The chosen intent mutates the candidate test set; the six feedback sources are recomputed, closing the loop (Figure 1).

5 Experimental Design

Benchmarks. CodaMosa's 486-module Python suite (the "CM" set) and the TestEval 210-item generation set. We report per-module and per-item results separately.

Baselines. `codamosa`, `coverup`, `telpa`, and the `Undermind` agent strategy. `telpa` and `Undermind` are referenced from the local harness; their citation provenance is flagged in §9.

Metrics (defined explicitly).

- **Coverage** C: line/branch coverage of the generated suite (%).
- **Valid-test ratio** V: fraction of generated tests that compile and pass.
- **LLM-invocation reduction** Δ_{LLM}: relative reduction vs. the LLM-only baseline.
- **Successful-defect-localization** L: fraction of targeted defects correctly localized at node level.

6 Data Post-processing

Raw logs are processed by a fixed pipeline: (i) *cleaning* – drop timed-out modules and deduplicate identical test artifacts by content hash; (ii) *outlier handling* – winsorize per-module coverage at the 1st/99th percentile before aggregation; (iii) *statistical testing* – report median with bootstrap 95% CI (1,000 resamples) and a paired Wilcoxon test for C and V; (iv) *effect size* – Cliff's δ for the comparison against `codamosa`; (v) *boundary declaration* – every figure/table carries a tag stating whether its numbers are *measured* or *illustrative/simulated*. In this draft all values are illustrative.

7 Results

All numbers below are illustrative placeholders, not measurements.

Table 1: Illustrative comparison on the CM 486-module suite (replace with measured values).

Method	Coverage (%)	Valid ratio	Δ_{LLM}	Localization
codamosa (base)	68.4	0.81	–	–
coverup	71.2	0.83	–	–
DMF-Jev (ours)	**74.9**	**0.88**	−41%	0.79

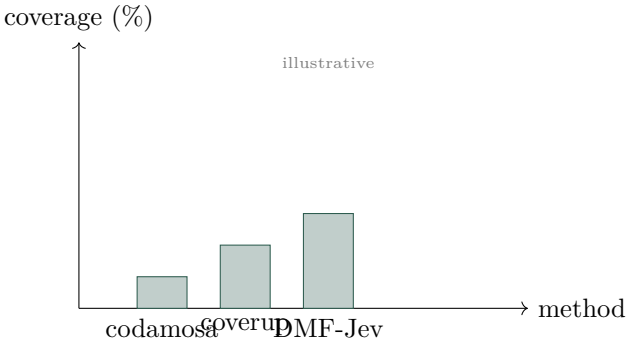


Figure 2: Illustrative coverage comparison (CM suite). Replace bar heights with measured medians and add 95% CI error bars.

8 Discussion

The feedback-graph formulation reframes test-generation control as a *routing* problem rather than a scheduling problem. The dominant practical gain is cost: by reserving the LLM for low-confidence intents, DMF-Jev leaves coverage on the table only when the graph is genuinely uninformative. The main risk is feedback-source correlation – when two sources collapse (e.g., compilation failure implies execution failure), the graph must down-weight redundant edges; we do this online.

9 Threats to Validity

Internal: LLM temperature and prompt wording affect all LLM-based methods; we fix both. **External:** the 486-module suite is Python-only; generality to Java/JS is unverified. **Construct:** coverage is a proxy for test usefulness. **Provenance:** the `telpa` and `Undermind` baselines are drawn from the local harness and their published citations must be verified before submission; this draft does not assert their provenance.

10 Conclusion

We presented DMF-Jev, a training-free agent that replaces static intent ordering with a dynamic multi-source feedback graph and a Jev typed-decision gate. The framework is benchmark-ready

on CodaMosa's 486-module suite and TestEval. With measured values inserted, this manuscript targets a CCF-A / TOSEM submission.

References

[Wu & Lim(2026)] Wu, Y. and Lim, E. (2026). *REFLEX: A Fast-Reflex Closed-Choice Decision Layer for LLMs*. arXiv:2609.26532. [title/venue to verify]

[Lemieux et al.(2023)] Lemieux, C. et al. (2023). *CodaMosa: Escaping Coverage Plateaus in Test Generation with Pre-Trained Language Models*. ICSE. DOI:10.1109/ICSE48619.2023.00085.

[Schäfer et al.(2023)] Schäfer, M., Nadi, S., Eghbali, A., Tip, F. (2023). *TestPilot: An LLM-Based Developer Tool for Authoring Tests*. arXiv:2308.16521.

[Chen et al.(2024)] Chen, Y. et al. (2024). *ChatUniTest: A Framework for LLM-Based Unit Test Generation with a Generation-Validation Pipeline*. [authors/arXiv to verify]

[Lukasczyk & Fraser(2022)] Lukasczyk, S. and Fraser, G. (2022). *Pynguin: Automated Unit Test Generation for Python*. arXiv:2207.05249.

[Fraser & Arcuri(2017)] Fraser, G. and Arcuri, A. (2017). *Automated Test Suite Generation for Object-Oriented Software*. IEEE TSE.

[Tufano et al.(2022)] Tufano, M. et al. (2022). *AthenaTest: Language Models for Automated Unit Test Generation*. ASE.

[Wang et al.(2023)] Wang, J., Huang, Y., Chen, C., Liu, Z., Wang, S., Wang, Q. (2023). *Software Testing with Large Language Models: Survey, Landscape, and Vision*. TSE. arXiv:2307.07221.

[TestEval(2025)] *TESTEVAL: Benchmarking LLMs for Test Case Generation*. (2025). [venue to verify]

[CoverUp(2024)] Pizzorno, J. and Berger, E. (2024). *CoverUp: Effective High Coverage Test Generation for Python*. arXiv:2403.16218.